48-Hour Intelligence Brief

AI Agents & Agentic AI — Multi-Agent Frameworks Landscape

October 2026 | $497 Fixed-Scope Deliverable | Investigative Cell

Scope & method

This brief answers one decision question: which multi-agent orchestration framework should a team build on today, and under what conditions? It covers LangGraph, CrewAI, AutoGen and its successor the Microsoft Agent Framework, plus adjacent options such as the OpenAI Agents SDK, Google ADK, n8n and no-code platforms. Each claim is weighed against the strongest available counterpoint, and remaining uncertainties are stated explicitly rather than smoothed over.

The method follows the standard Investigative Cell format: a direct answer first, then the evidence behind it, the best arguments against it, what is still unknown, and concrete recommendations. Vendor-reported figures are labelled as such and are not treated as independent evidence.

Contents

Section	Page
1. Direct Answer	2
2. Key Evidence (framework profiles and comparison)	3–4
3. Strongest Counterpoints	5
4. Residual Uncertainties	6
5. Actionable Recommendations and decision matrix	7
6. Source Notes	8

1. Direct Answer

The multi-agent framework landscape in late 2026 is dominated by LangGraph for production-grade, stateful, auditable systems and CrewAI for rapid role-based prototyping and business workflows. AutoGen has been placed in maintenance mode by Microsoft; its successor is the Microsoft Agent Framework. OpenAI Swarm is effectively discontinued in favor of the OpenAI Agents SDK. Production adoption estimates place LangGraph at roughly 38% of multi-agent deployments, with custom orchestration still significant and CrewAI trailing in production maturity despite strong demo velocity.

What this means in practice

The market has split along a clear axis: **control versus speed**. LangGraph models an agent system as an explicit graph of nodes and edges with persistent state, which makes behaviour inspectable and replayable but asks more of the engineering team up front. CrewAI models the same problem as a crew of role-playing agents with tasks, which is faster to express and easier for non-specialists to read, but gives less fine-grained control over execution paths.

The AutoGen situation is the most important change for teams with existing investments. A framework in maintenance mode still works, but it will not keep pace with new model capabilities, security fixes may slow, and community knowledge will drift toward the successor. Teams should treat AutoGen code as a migration candidate rather than a foundation.

Custom orchestration remaining significant is itself a signal. Many teams find that a thin, in-house control loop around a model API is easier to reason about than a general framework, particularly when the workflow is narrow and well understood.

Bottom line

- Choose LangGraph when the workflow is long-running, branching, regulated or needs human sign-off.
- Choose CrewAI when the goal is to prove value quickly with role-based workflows.
- Do not start new production work on AutoGen or OpenAI Swarm.
- Keep custom orchestration on the table for narrow, stable workflows.

2. Key Evidence

2.1 LangGraph

LangGraph: Highest production footprint. Explicit stateful graphs, first-class checkpointing, human-in-the-loop interrupts, and mature LangSmith observability. Approximately 71 million monthly PyPI installs. Best for long-running, auditable, branching workflows. Pricing: free library + LangSmith Plus at $39/seat/month.

Why it matters: checkpointing lets a workflow pause, survive a restart and resume from a known state, which is what makes human approval steps and long-running jobs practical. The graph structure also makes it possible to explain after the fact which path an agent took and why, a recurring requirement in audits and incident reviews. The trade-off is that developers must think in terms of state schemas, nodes and transitions rather than simply describing agents.

2.2 CrewAI

CrewAI: Fastest for role-based crews (researcher, writer, reviewer). Claims 450 million+ agentic workflows per month and strong enterprise features (FedRAMP, VPC, SSO). Approximately 20 million monthly installs. Ideal for prototypes and business process automation. Free core + limited free hosted executions; enterprise custom.

Why it matters: the role-and-task abstraction maps directly onto how business teams already describe work, so a working prototype can often be assembled and demonstrated quickly. The higher-level abstraction hides some execution detail, which helps early on but can make debugging, cost control and deterministic behaviour harder as workflows grow. Enterprise features are relevant for buyers with compliance requirements, but should be confirmed against the specific deployment model being purchased.

2.3 AutoGen and the Microsoft Agent Framework

AutoGen: Maintenance mode since approximately October 2025. No major Python releases in hundreds of days. Microsoft directs users to Microsoft Agent Framework (GA April 2026). Still referenced in research but not recommended for new production builds.

Why it matters: AutoGen popularised conversational multi-agent patterns and remains common in academic and tutorial material, so teams will keep encountering it. Its successor is the supported path inside the Microsoft ecosystem and is most attractive to organisations already standardised on Microsoft cloud, identity and tooling.

2.4 Other options

Others: Microsoft Agent Framework (successor path), OpenAI Agents SDK, Google ADK, n8n, and various no-code platforms. Custom orchestration remains approximately 28% of production deployments.

Vendor SDKs such as the OpenAI Agents SDK and Google ADK tend to offer the smoothest experience with their own models and hosted tools, at the cost of tighter coupling to one provider. Workflow tools such as n8n and no-code platforms suit integration-heavy automation where agents are one step among many, but are usually not the right home for complex multi-agent reasoning loops.

2.5 Side-by-side comparison

Dimension	LangGraph	CrewAI	AutoGen / MS Agent Framework	Custom orchestration
Core abstraction	Stateful graph of nodes and edges	Crew of role-based agents and tasks	Conversational agents / successor framework	Team-defined control loop
Strength	Control, auditability, resumability	Speed to prototype, readability	Microsoft ecosystem fit (successor)	Full control, minimal dependencies
Human-in-the-loop	First-class interrupts	Supported at a higher level	Varies; check successor docs	Whatever the team builds
Observability	Mature (LangSmith)	Platform features; verify depth	Microsoft tooling (successor)	Must be built or bought
Learning curve	Steeper	Gentle	Moderate	Depends on team
Lifecycle status	Actively developed	Actively developed	AutoGen: maintenance mode	Owned by team
Best fit	Production, regulated, long-running	Prototypes, business workflows	Existing Microsoft-centric shops	Narrow, stable workflows

Reading the evidence

The install figures indicate developer interest and are useful for gauging community size, documentation depth and hiring pool, but downloads include CI pipelines and transitive dependencies and are not a count of production systems. The capability differences in the table are more decision-relevant than any single adoption number: the right choice depends on whether the workflow needs determinism and auditability or speed and flexibility.

3. Strongest Counterpoints

CrewAI's "Fortune 500 / 450M workflows" figures are self-reported; independent production adoption data places it well behind LangGraph. LangGraph's production advantage comes with a steeper learning curve and potential operational costs (checkpoint storage growth in Postgres is a documented issue). Many "AI agent platforms" in marketing materials are thin wrappers or no-code tools rather than true multi-agent orchestration frameworks. Pricing units vary widely. Market estimates for production share are directional, not audited census data.

3.1 Against the LangGraph default

Control has a price. Teams new to graph-based state machines may spend longer reaching a first working version, and the operational surface (checkpoint stores, retention policies, tracing) must be owned by someone. For a short, linear workflow, this can be more machinery than the problem requires.

3.2 Against CrewAI as a starting point

Prototypes have a habit of becoming production systems. If the migration path to a more controlled architecture is never funded, a team can end up running critical processes on an abstraction chosen for demo speed. Self-reported scale figures do not answer whether the framework fits a specific reliability requirement.

3.3 Against frameworks altogether

A credible view holds that frameworks add abstraction faster than they add value, and that a small, well-tested control loop around a model API is easier to debug and cheaper to run. The significant share of custom orchestration supports this view for narrow use cases, though it shifts maintenance burden entirely onto the team.

3.4 Against the evidence base

Much of the available data comes from vendors, download counters and marketing material. Category labels are loose: products marketed as agent platforms range from genuine orchestration frameworks to simple prompt wrappers. Comparisons across pricing models (per seat, per execution, enterprise contract) are rarely like-for-like.

4. Residual Uncertainties

Exact enterprise revenue and true production deployment counts for each framework remain vendor-influenced. Rapid consolidation continues. The landscape can shift again within 3–6 months. Real-world failure modes and long-term maintenance costs are under-documented outside of selective case studies.

4.1 What we cannot confirm

- How many production systems each framework actually runs, as opposed to downloads, trials or demos.
- Revenue and customer counts, which are largely disclosed by vendors themselves.
- How the Microsoft Agent Framework will be adopted by former AutoGen users over the next release cycles.
- Long-run cost behaviour of state persistence, tracing and model usage at scale.

4.2 What could change the answer

- Consolidation: acquisitions, deprecations or licence changes can alter the field quickly, as the AutoGen and Swarm transitions show.
- Model-provider SDKs absorbing orchestration features, reducing the need for a separate framework.
- Emerging interoperability standards for tools and agent-to-agent communication, which could lower switching costs.
- Security incidents or regulatory guidance that raise the bar for auditability and human oversight.

4.3 How to reduce the uncertainty

Treat this brief as a starting position, not a verdict. Before committing, check current release activity and deprecation notices for the shortlisted frameworks, request reference customers with comparable workloads, and run a time-boxed pilot that records cost, latency, failure rate and debugging effort on your own data. Revisit the decision on a regular cadence given how fast this market is moving.

5. Actionable Recommendations

1. For production systems needing control, auditability, and human approval → Default to LangGraph.
2. For rapid internal prototypes or role-based business workflows → Start with CrewAI, then migrate critical paths if needed.
3. Avoid new builds on AutoGen; evaluate Microsoft Agent Framework only if already deep in the Microsoft stack.
4. Always calculate total cost of ownership (framework + observability + model API + storage + engineering time).
5. Validate with a narrow pilot on your actual workflow before committing.

5.1 Decision matrix

If your situation is…	Lean toward	Watch out for
Regulated or audited workflow requiring human approval	LangGraph	Checkpoint storage growth; team ramp-up time
Need a working demo for stakeholders quickly	CrewAI	Prototype silently becoming production
Existing AutoGen code in use	Plan migration (MS Agent Framework or alternative)	Running unsupported code in critical paths
Organisation standardised on Microsoft stack	Microsoft Agent Framework	Maturity of a recently released framework
Committed to a single model provider	That provider's SDK (e.g. OpenAI Agents SDK, Google ADK)	Vendor lock-in and portability
Narrow, stable, well-understood workflow	Custom orchestration	Owning observability and maintenance
Integration-heavy automation, light reasoning	n8n or no-code platforms	Limits on complex multi-agent logic

5.2 Pilot checklist

- Pick one real workflow with a clear success measure.
- Track cost per run, latency, failure rate and time spent debugging.
- Test a human-approval step and a mid-run failure and resume.
- Record total cost of ownership, not just licence or seat price.

6. Source Notes

A full Investigative Cell brief links every material claim to a primary or best-available source. This sample describes the categories of sources used rather than reproducing links.

Source categories a full brief would cite

- **Official documentation** for each framework: architecture, feature lists, lifecycle and deprecation notices.
- **Package download statistics** from PyPI, used as an indicator of developer interest, not production adoption.
- **Vendor release notes and announcements**, including general-availability and maintenance-mode statements.
- **GitHub repository activity**: release cadence, commit history, open issues and maintainer responses.
- **Vendor pricing pages** for observability and hosted offerings, checked on the date of the brief.
- **Independent surveys and practitioner reports** for production-share estimates, flagged as directional.

Evidence grading

Claims are graded by source type: vendor-reported figures are labelled as self-reported; download counts are treated as interest signals; production-share estimates are treated as directional. Where sources conflict, the conflict is shown in the Counterpoints section rather than resolved silently.

Important notice

The figures in this sample are illustrative and should be verified against current primary sources before being relied on for any decision. Numbers, dates and prices in this market change quickly. No source links are included in this sample.

This is a sample deliverable matching the $497 48-Hour Intelligence Brief format. Concise, source-linked, challenge-tested, and decision-oriented. Prepared by the Investigative Cell.